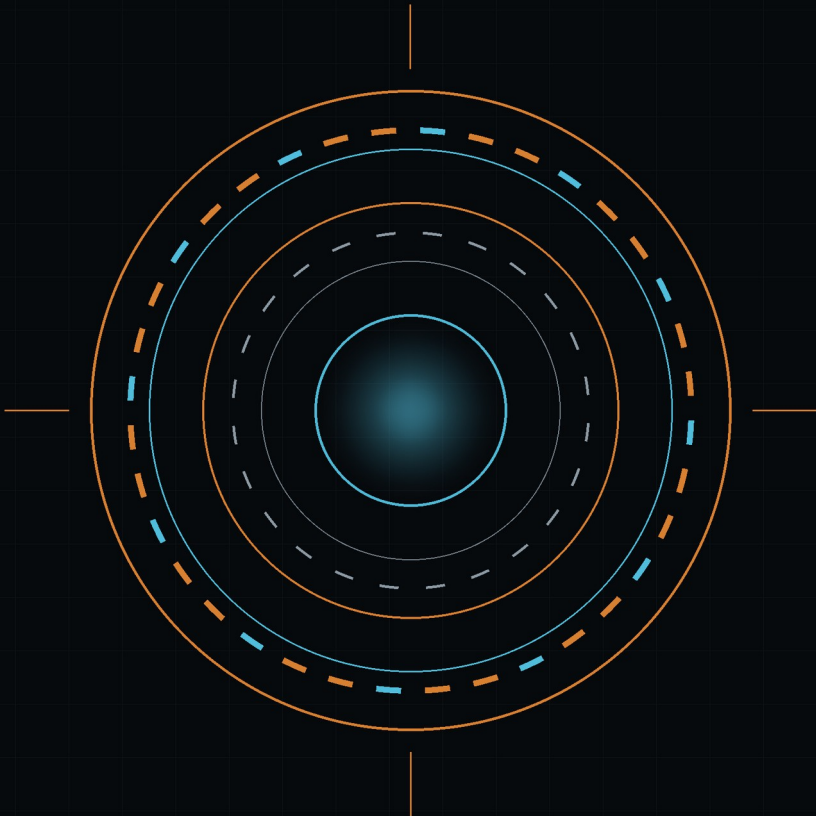


PROJECT J.A.R.V.I.S.

MASTER BLUEPRINT 1.0

PERSONAL AI OPERATING SYSTEM



SYSTEM ARCHITECTURE • VOICE • MEMORY • AGENTS • DEVICES
HUD • WORLD INTELLIGENCE • HOME • SECURITY • ROADMAP

MASTER SPECIFICATION // 19 AUGUST 2026

START HERE

Dieses Dokument ist die Source-of-Truth-Spezifikation für dein persönliches JARVIS-System.

ZWECK Wenn du diese PDF später wieder in einen Chat hochlädst, soll sie genügen, damit ein Assistent sofort versteht, was gebaut wird, warum die Architektur so aussieht und welcher nächste Entwicklungsschritt ansteht.

Das Ziel ist kein Chatbot und keine Film-Demo. Ziel ist ein installierbares, privates, geräteübergreifendes Personal-AI-Operating-System: Voice, Memory, Claude-Agenten, PC-Steuerung, Smart Home, Mobile, Remote-Zugriff, ein dynamisches futuristisches HUD, World-Intelligence-Globus, Live-Coding, Proaktivität und kontrolliertes persönliches Lernen.

Die fünf nicht verhandelbaren Prinzipien

- JARVIS Core ist das Produkt. Claude ist ein austauschbarer Intelligence Provider - nicht das gesamte System.
- Rechte, Sicherheit, Memory, Geräteidentität und Tool-Ausführung werden deterministisch von unserer Software kontrolliert; niemals nur durch einen Prompt.
- Local-first: persönliche Daten, Event-State, Geräteinformationen und langfristiges Memory bleiben möglichst zuhause bzw. auf deinen Geräten.
- Fluid-first: Wake Word, HUD, lokale Aktionen und Statusfeedback dürfen nie auf langsames Cloud-Reasoning warten.
- Build core before spectacle: zuerst Gehirn, Permissions, Tools und Memory; danach Voice/HUD; erst dann 3D-Globus, Alexa, Self-Extension und R&D-Features.

Wichtige Klarstellung zu Claude Max 20x

Max 20x ist vor allem ein höheres Nutzungskontingent, nicht eine eigene „Intelligenzstufe“. Für maximale Modelleleistung wählen wir innerhalb von Claude das stärkste geeignete Modell und dessen Effort-Level. Stand 19.08.2026 ist Claude Opus 5 für komplexes agentisches Coding verfügbar; Anthropic dokumentiert Effort von low bis max, wobei max für Aufgaben gedacht ist, bei denen maximale Leistungsfähigkeit zählt. Max 20x enthält Claude Code und bietet 20x das Pro-Kontingent pro 5-Stunden-Sitzungsfenster. [S1][S2]

Für die Entwicklung nutzen wir deshalb Claude Code + Opus 5 (xhigh/max) für Architektur, Security, Datenmodelle, schwierige Refactorings und Release-Reviews. Für Routine-Implementierung können Sonnet 5 oder niedrigere Effort-Stufen schneller und ressourcenschonender sein. Der Runtime-Core von JARVIS muss trotzdem einen Model Router besitzen, damit wir Anbieter und Modelle später austauschen können.

1. PRODUKTVISION UND DEFINITION

1.1 Ein-Satz-Definition

MASTER DEFINITION JARVIS ist ein persistenter, multimodaler, proaktiver persönlicher KI-Agent mit eigenem Core, langfristigem Memory, kontrollierter Tool- und Geräteausführung, Multi-Agent-Orchestrierung und dynamischer Präsenz auf Desktop, Handy und Zuhause.

1.2 Was JARVIS später können soll

Bereich	Zielzustand
Voice	„Jarvis“ wird lokal erkannt; sofortiges Feedback; flüssige Duplex-Konversation; Barge-in; dynamische Stimme; kein abgehacktes TTS.
PC	PC per Wake-on-LAN/Out-of-band-Option starten, Apps/Dateien/Terminal/Browser steuern, Bildschirm sehen, Aufgaben ausführen und Ergebnis verifizieren.
Mobile	Unterwegs sprechen, Zuhause steuern, Missionen verfolgen, Bestätigungen freigeben, Live-HUD sehen, Kamera optional als Sensor verwenden.
Smart Home	Home Assistant als Geräte-Nervensystem: Licht, Klima, TV, Steckdosen, Sensoren, Kameras, Szenen und Routinen.
Alexa/Echo	Als zusätzlicher Satellit/Bridge nutzen, soweit Amazons Plattform es erlaubt; nicht als Kern und nicht mit dem Versprechen, Alexa OS/Wakeword vollständig zu ersetzen.
Memory	Episodisch, semantisch, Projekt-, Präferenz-, Habit-, Relationship- und Procedural-Memory mit Quellen, Confidence und Lösbarkeit.
Learning	Gewohnheiten, Begriffe, Arbeitsstil und Präferenzen lernen; Korrekturen verwerten; neue Routinen vorschlagen; keine unkontrollierte Core-Selbstmodifikation.
Coding	Live sehen, welche Dateien entstehen, welche Diffs gemacht werden, Builds/Tests verfolgen, Preview starten und Fehler iterativ beheben.
World Intelligence	Interaktive 3D-Erde: News nach Land/Region, rotierende Kamera, Marker, Themenlayer, Quellen/Confidence und Zeitachse.
UI	Dynamisches futuristisches Command-Center mit AI-Core, Amber/Cyan-Akzenten, kontextabhängigen Modi und 60/120-FPS-Ziel.
Personality	Ruhig, souverän, präzise, trockener Sarkasmus, dosierter schwarzer Humor, kaum Füllsätze, ernst in Notfällen, adaptive Gesprächslänge.
Privacy/Security	Private Daten lokal, verschlüsselte Verbindungen, device-bound identities, passkey/biometric approvals, Audit Logs, Sandbox, Rollback und Kill Switch.

1.3 Was ausdrücklich NICHT das Ziel ist

- Keine 1:1-Kopie der Stimme eines realen Schauspielers. Wir bauen eine eigene hochwertige Stimme mit ähnlichen Eigenschaften: britisch/elegant, ruhig, trocken, klar.
- Keine Behauptung von Bewusstsein oder echter AGI. Das System ist ein Agenten-OS mit Modellen, Tools, Memory und Automatisierung.

- Keine unkontrollierte Selbstmodifikation, keine eigenmächtige Rechteausweitung, kein automatisches Root/Admin ohne Policy Engine.
- Keine Abhängigkeit von einem einzigen Cloud-Anbieter oder von einem einzelnen UI-Framework.
- Keine UI-Spielerei auf Kosten von Zuverlässigkeit, Latenz oder Sicherheit.

2. ZIELERLEBNIS: SO SOLL SICH JARVIS ANFÜHLEN

2.1 Wake / Presence

Du sagst „Jarvis“. Innerhalb eines sehr kurzen Augenblicks reagiert das aktive Gerät akustisch und visuell. Der Core entscheidet, welches Display/Lautsprecher gerade der beste Ausgabepunkt ist. Am Desktop materialisiert sich der AI-Core; am Handy öffnet sich eine reduzierte HUD-Ansicht; in einem Raum kann ein Voice-Satellit übernehmen. Andere Geräte bleiben still, außer eine Mission verlangt bewusst Multi-Screen-Output.

2.2 Beispiel: von unterwegs den PC benutzen

DU: "Jarvis, schalte meinen PC zuhause ein, öffne Projekt Atlas und prüfe den letzten Build."

```
Mobile Wake -> Home Core -> Wake-on-LAN -> Desktop Agent online
-> Project Context laden -> Build ausführen -> Fehler/Erfolg verifizieren
-> Live-Status ans Handy -> Ergebnis sprechen/anzeigen
```

Falls der PC nicht per WOL erreichbar ist, kann die Architektur später robustere Out-of-band-Optionen integrieren (z.B. Intel AMT/IPMI auf geeigneter Hardware). Ein Smart Plug allein ist kein generischer Ersatz für sauberes Power Management.

2.3 Beispiel: News / World Intelligence

Du fragst nach aktuellen Nachrichten. Der News-Agent dedupliziert Meldungen, extrahiert Länder/Regionen, bewertet Quellen und liefert ein strukturiertes Ereignis-Set. Die UI dreht die 3D-Erde zum jeweiligen Land, hebt es hervor und synchronisiert Marker, Karten und Sprache. Der Globus ist nicht nur Animation - er ist ein interaktiver Datenraum mit Themen-Layern, Zeitachse und Evidence View.

2.4 Beispiel: „Jarvis, programmiere ein Spiel“

```
MISSION: BUILD GAME
Planning -> Project scaffold -> Files created -> Code diffs -> Build -> Test
-> Bug found -> Patch -> Rebuild -> Run -> Live Preview -> Verify
```

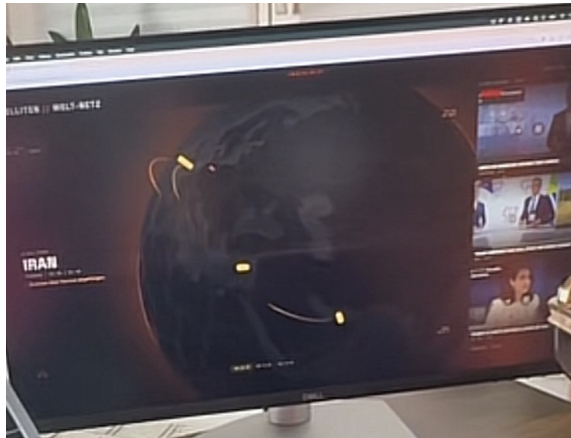
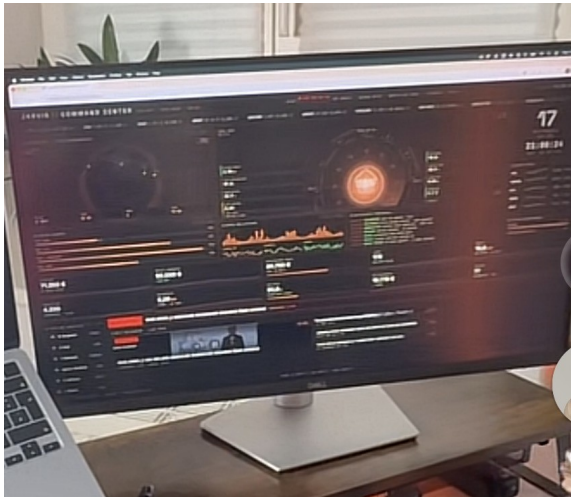
```
HUD shows:
• current file / diff
• terminal output
• test status
• agent activity
• preview window
• final artifact
```

Die Oberfläche zeigt keine versteckten privaten Reasoning-Ketten des Modells. Sie zeigt stattdessen nachvollziehbare Arbeit: Pläne auf hoher Ebene, Tool-Aufrufe, Dateidiffs, Tests, Logs, Artefakte, Verifikationsstatus und nächste Schritte.

3. VISUELLE PRODUKTSPRACHE

3.1 Referenzrichtung aus deinen Bildern

Die folgenden zwei Referenzbilder definieren die Designrichtung: dunkles Command-Center, technische Mikrodetails, zentrale Fokusobjekte, Amber/Orange plus Cyan, kreisförmige Radar-/Orbit-Motive, transparente Ebenen und dichte, aber hierarchisch klare Datenvisualisierung. Wir kopieren kein Film-UI 1:1; wir entwickeln daraus eine eigenständige JARVIS-Identität.



Designreferenzen aus dem Projektchat: dunkles HUD, rotierende Kerne/Ringe, feine Datenebenen und futuristische Systemarchitektur.

3.2 Dynamische HUD-Modi

Befehl / Situation	Adaptive Ansicht
„Jarvis“	Central AI Core: Listening / Thinking / Speaking / Working.
News	3D World Intelligence Globe + country cards + source/evidence rail.
Coding	Monaco Editor + tree + diff + terminal + tests + live preview.
Smart Home	2D/3D Digital Twin des Hauses + Räume + Geräte + Sensorstatus.
System	PC/Netzwerk/Service topology + load + health + alerts.
Research	Source graph, evidence cards, contradictions, timeline.
Mission	Goal tree, agents, dependencies, approvals, checkpoints, ETA/health.
Idle	Minimaler Core / Ambient mode - keine unnötigen Panels.

3.3 Performance-Regel für das HUD

FLUIDITY RULE Das UI läuft getrennt vom AI-Backend. Keine Claude-Anfrage darf Animation, Eingabe, Scrollen, Globe-Rendering oder Wake-Feedback blockieren. Ziel: 60 FPS Mindestgefühl, 120 FPS auf geeigneter Hardware; WebGPU/WebGL-Fallback für 3D. Three.js WebGPURenderer kann WebGPU nutzen und auf WebGL 2 zurückfallen. [S10]

4. GESAMTARCHITEKTUR

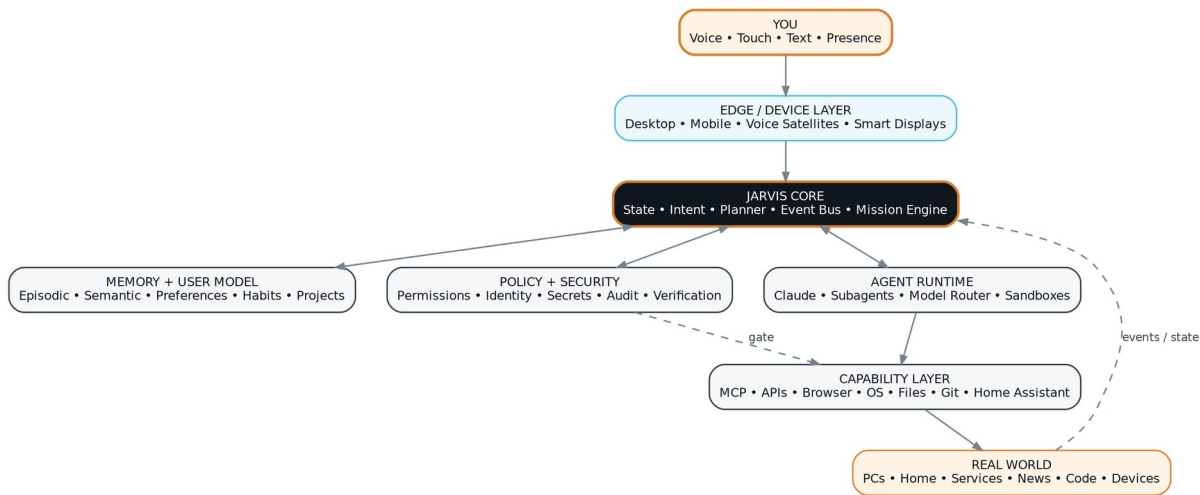


Abbildung 1 - Zielarchitektur: Core und Security zwischen Modell und realer Welt.

4.1 Zentrale Designentscheidung: Claude ist nicht JARVIS

JARVIS ist die Kombination aus Core, State, Event Bus, Memory, Policy Engine, Agent Runtime, Tool Registry, Device Mesh, Voice und UI. Claude liefert Reasoning und agentische Planung. Diese Trennung verhindert Vendor Lock-in und macht lokale Fallbacks, schnellere Modelle und zukünftige Modellwechsel möglich.

4.2 Modularer Monolith zuerst

Version 0.x wird bewusst kein Microservice-Zoo. Ein Python-Core-Prozess enthält klar getrennte Module und stabile Interfaces. Voice, Desktop und UI können separate Prozesse sein. Erst wenn Last, Isolation oder Geräteverteilung es erfordern, werden Services herausgezogen. So reduzieren wir Komplexität, ohne spätere Skalierung zu verbauen.

4.3 Empfohlener Tech Stack

Schicht	Empfehlung	Warum
Core	Python 3.12 + asyncio	Stark für Agenten, lokale AI/Audio-Ökosysteme, schnelle Iteration.
Agent Runtime	Claude Agent SDK (Python zunächst)	Programmierbare Agent-Schleife, Tools, Berechtigungen, Kosten-/Ausgabekontrolle; gleiche Grundmechanik wie Claude Code. [S3]
Desktop/Mobile UI	Tauri 2 + React + TypeScript	Ein UI-Codebestand für Windows/macOS/Linux/Android/iOS; native Integrationen via Rust/Swift/Kotlin möglich. [S9]
3D HUD	Three.js / React Three Fiber + WebGPU/WebGL fallback	Globus, AI-Core, Partikel, 3D-Haus und technische Visualisierungen. [S10]
Code View	Monaco Editor	VS-Code-artiges Live-Editing, Diffs und Syntax Highlighting.
Database	PostgreSQL + pgvector	State, Missionen, Memory-Metadaten, Vektor-Retrieval und Multi-Device-Zugriff.
Local cache	In-process cache -> optional Redis/NATS später	Schneller Start; später verteilte Events/Queues möglich.
Browser	Playwright + API-first adapters	Deterministischere Browserautomatisierung, wenn keine API existiert.

Home	Home Assistant	Breite Geräteintegration, lokale Voice-/Wake-Word-Möglichkeiten und Automationen. [S8][S9A]
Tool protocol	MCP + interne Capability API	Standardisierte Tool-Anbindung; MCP 2026-07-28 stärkt Statelessness, Routing und Authorization. [S7]
Packaging	Tauri bundler + signierte Installer	Später EXE/MSI, DMG, Linux Bundle und Mobile Builds.
Repo/CI	Private GitHub + Actions + Releases	Private Source-of-Truth, Tests, Build-Artefakte, Releases. [S11][S12]

5. DAS TECHNISCHE GEHIRN: JARVIS CORE

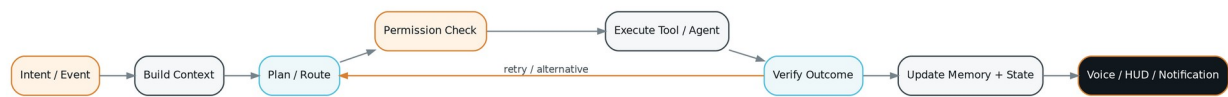


Abbildung 2 - Jede Aktion durchläuft Kontext, Planung, Permission, Ausführung, Verifikation und Memory.

5.1 Core-Module

Modul	Verantwortung
State Manager	Aktueller Benutzer-, Geräte-, Mission-, Gesprächs- und Präsenzzustand.
Intent Router	Schnelle lokale Befehle von komplexen Reasoning-Aufgaben unterscheiden.
Context Builder	Nur den relevanten Kontext aus Memory, App-Zustand, Projekt und Sensoren zusammenstellen.
Mission Engine	Lang laufende Ziele in Tasks, Dependencies, Checkpoints und Status umwandeln.
Planner	Schritte und Agenten bestimmen; Budget und Risiken berücksichtigen.
Agent Coordinator	Claude/Subagents starten, stoppen, begrenzen, Ergebnisse zusammenführen.
Capability Registry	Alle Tools/Skills mit Schema, Risiko, benötigten Rechten und Health registrieren.
Permission Engine	Vor jeder Aktion deterministische Policy prüfen.
Execution Gateway	Tool-Aufrufe ausführen - nie direkter Modellzugriff auf OS/Secrets.
Verifier	Prüfen, ob die reale Aufgabe tatsächlich erfolgreich war.
Event Bus	Alle Zustandswechsel typisiert an HUD, Mobile, Logs, Memory und Automation verteilen.
Scheduler	Zeitbasierte Jobs, Background Missions, Retry/Backoff.
Model Router	Opus/Sonnet/lokal je nach Schwierigkeit, Latenz, Privatsphäre und Kosten.
Audit Logger	Nachvollziehbare, manipulationsgeschützte Historie kritischer Aktionen.

5.2 Core Event Envelope

```
Event {
  event_id: UUID
  type: "mission.task.started"
  timestamp: ISO-8601
  source: "coding-agent"
  correlation_id: UUID
  user_id: local-owner
  device_id: optional
  sensitivity: public | private | secret
  priority: background | normal | urgent | critical
  payload: {...}
  ttl: optional
}
```

5.3 Mission State Machine

```
CREATED -> PLANNING -> WAITING_FOR_APPROVAL -> RUNNING
          -> VERIFYING -> COMPLETED
```

-> PAUSED / BLOCKED / FAILED / CANCELED

Every transition emits an event and is persisted.

5.4 Definition of Done für Core 0.1

- ☐ Textkommando erreicht Core über lokale API/WebSocket.
- ☐ Intent Router wählt zwischen lokalem Mock-Tool und Claude-Agent.
- ☐ Permission Engine blockiert/erlaubt/fordert Bestätigung.
- ☐ Tool Registry führt mindestens drei Mock-Tools aus.
- ☐ Jede Aktion erzeugt Events und Audit-Logs.
- ☐ Mission bleibt nach Prozessneustart erhalten.
- ☐ Verifier unterscheidet „Tool wurde aufgerufen“ von „Ziel erreicht“.
- ☐ Keine UI außer minimalem Debug-Dashboard ist für diesen Meilenstein nötig.

6. CLAUDE-STRATEGIE: MAXIMALE INTELLIGENZ, OHNE DEN CORE DAVON ABHÄNGIG ZU MACHEN

6.1 Modellstrategie Stand 19.08.2026

Claude Opus 5 wurde am 24.07.2026 veröffentlicht; Anthropic beschreibt es als starken Sprung bei tiefem Reasoning, agentischem Coding, Long-Horizon-Aufgaben und Multi-Agent-Koordination. Es besitzt ein 1M-Token-Kontextfenster, bis zu 128k Output-Tokens und Effort low/medium/high/xhigh/max. [S2][S2A]

Aufgabe	Empfohlenes Modell/Setting	Anwendung
Architektur / Security / Datenmodell	Opus 5, effort=max	Vor Implementierung planen, danach kritisch reviewen.
Schwierige Refactorings / Multi-Agent	Opus 5, xhigh oder max	Lange Tool-Schleifen, viele Dateien, komplexe Abhängigkeiten.
Normale Feature-Implementierung	Sonnet 5 high/xhigh oder Opus 5 high	Schnelleres Kosten-/Latenz-Verhältnis.
Routing / Klassifikation	kleines/schnelles Modell oder lokale Regeln	Keine teure Frontier-KI für triviale Entscheidungen.
Offline / private Basics	lokales Modell + deterministic intents	Home/PC-Grundkommandos bleiben funktionsfähig.

6.2 Claude Agent SDK

Anthropics Agent SDK stellt die Agent-Schleife von Claude Code als Python-/TypeScript-Bibliothek bereit: Dateien lesen, Befehle ausführen, Code editieren, Web verwenden und Tool-/Permission-Kontrolle programmatisch steuern. [S3][S4] Für JARVIS ist das ein starker Baustein, aber wir kapseln ihn hinter unserem eigenen `IntelligenceProvider` und `AgentRuntime` Interface.

BILLING / AUTH Für eine echte Anwendung sollten API-/SDK-Authentifizierung und Kosten sauber getrennt vom Chat-Abo behandelt werden. Anthropics aktuelle Dokumentation nennt für berechnigte Max-20x-Nutzer ein separates monatliches Agent-SDK-Guthaben; Planbedingungen können sich ändern. Architektur niemals auf ein bestimmtes Abo-Detail festnageln. [S1A][S4]

6.3 Multi-Agent-Pattern

Anthropic unterstützt heute kontext-isolierte Multi-Agent-Orchestrierung. Unser Standardpattern für schwierige Aufgaben wird Writer -> Verifier -> Security/QA sein, statt zehn Agenten ohne Not zu starten. Parallele Agenten lohnen sich nur, wenn Teilprobleme wirklich unabhängig sind. [S5]

```
Coordinator
├── Research Agent
├── Implementation Agent
├── Test Agent
└── Verification Agent

Security-sensitive task:
Coordinator -> Implementer -> Security Reviewer -> Permission Engine -> Executor
```

7. SECURITY, RECHTE UND VERTRAUEN

7.1 Permission-Level

Level	Klasse	Beispiele	Standard
P0	Observe	Bildschirmstatus, Gerätezustand, Read-only Sensoren	Automatisch, protokolliert
P1	Safe	App öffnen, Licht schalten, lokale Suche	Automatisch
P2	Reversible	Datei verschieben, Fenster anordnen	Automatisch + Undo/Log
P3	Sensitive	E-Mail senden, Kalender ändern, Nachricht senden	Bestätigung je Kontext
P4	Critical	Software installieren, Deploy, Shutdown mit laufenden Jobs	starke Bestätigung / biometrisch
P5	Restricted	Secrets, Admin-/Security-Einstellungen	spezielle Policies, oft device-bound
P6	Forbidden	explizit gesperrte Aktionen	nie ausführen

7.2 Harte Sicherheitsregeln

- Voice-ID ist Komfortsignal, kein alleiniger Authenticator. Kritische Freigaben über entsperartes vertrauenswürdigen Gerät, Passkey oder Biometrie.
- Secrets werden über Credential Broker/OS Keychain/Vault genutzt; das Modell soll Schlüssel möglichst nie im Klartext sehen.
- Agenten bekommen pro Mission temporäre Capabilities; Rechte laufen ab.
- Shell/OS-Aktionen laufen in Sandbox oder über allowlisted Adapter; keine „Claude hat einfach Admin“ Architektur.
- PreToolUse/Hook-ähnliche Gates blockieren gefährliche Aufrufe deterministisch. Claude Code dokumentiert Hooks, die Tool-Aufrufe vor Ausführung blockieren oder auf Ask setzen können. [S6]
- Jeder kritische State Change bekommt Audit Event, vorherigen Zustand und möglichst Rollback-Punkt.
- Globaler Kill Switch: „Jarvis, stop everything“ plus UI-/Hardware-Schalter; der Core stoppt aktive Agenten unabhängig vom Modell.
- Remote-Zugriff nie über offen ins Internet gestellte Admin-Ports; privater Mesh-VPN/WireGuard oder gleichwertiger Zero-Trust-Tunnel.

7.3 Threat Model - wichtigste Risiken

Risiko	Gegenmaßnahme
Prompt Injection aus Webseiten/Dateien	untrusted content labels, tool scope, allowlists, verifier, keine secrets im prompt
Fehlinterpretierter Voice-Befehl	Confidence + risk-based confirmation + device context
Falsches „fertig“	independent verifier + outcome checks
Compromised Smart Device	Netzsegmentierung/VLAN, Home Assistant gateway, begrenzte capabilities
Gestohlenes Handy	OS lock, device key revocation, passkeys, remote revoke
Agent-Endlosschleife	time/token/cost budgets, watchdog, checkpoints
Datenverlust	versionierte verschlüsselte Backups, snapshots, restore drills
UI täuscht Status vor	UI rendert nur server-signierte/persistierte Events, nicht frei erfundene Animationen

8. MEMORY UND PERSONALISIERUNG

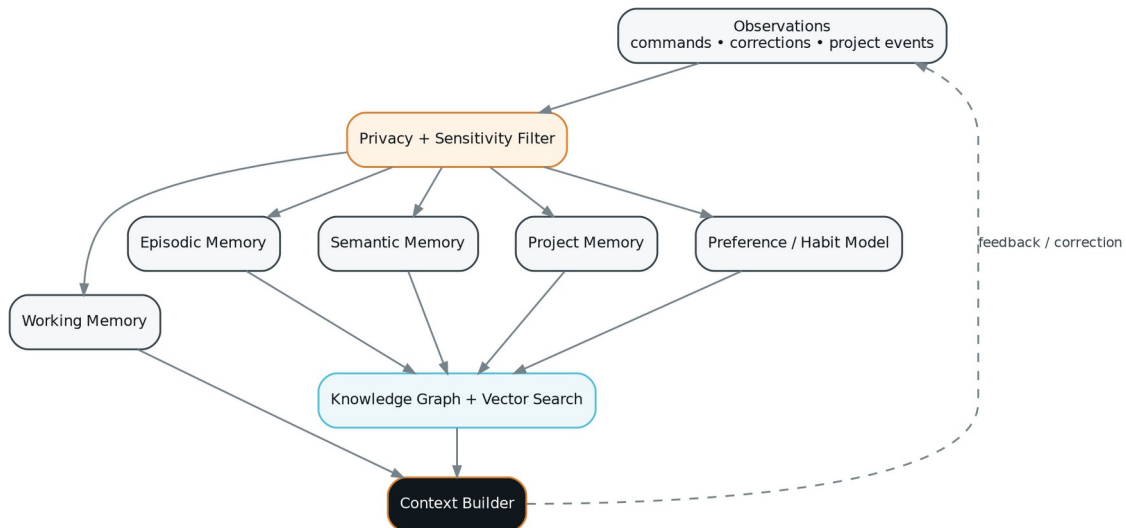


Abbildung 3 - Memory-Pipeline mit Privacy Filter, strukturierten Stores und Context Builder.

8.1 Memory-Typen

Typ	Beispiel
Working	„Wir debuggen gerade `voice_router.py`.“
Episodic	„Am 19.08. wurde Build 0.3.4 nach einem Audio-Deadlock repariert.“
Semantic	„Projekt Atlas verwendet PostgreSQL.“
Project	Architektur, offene Tasks, Entscheidungen, Dateien, Deploy-Verfahren.
Preference	Antwortlänge, IDE, Designstil, bevorzugte Apps.
Habit	Wiederkehrende Sequenzen und Tagesmuster.
Relationship	Personen/Firmen/Projekte und Beziehungen.
Procedural	„So deployen wir Projekt X.“
Visual optional	relevante Screens/Layouts, nur nach Datenschutzregeln.

8.2 Ein Memory-Eintrag

```

memory_id: uuid
type: preference
subject: owner
predicate: preferred_editor
value: VS Code
confidence: 0.94
source: explicit_statement | observation | correction
observations: 14
created_at: ...
last_confirmed_at: ...
sensitivity: private
retention: durable
project_scope: optional
  
```

8.3 Lernschleife

Beobachtung -> Muster -> Hypothese -> Confidence -> sichere Personalisierung -> Feedback. Wiederholte Verhaltensmuster dürfen zunächst Vorschläge erzeugen; erst nach Freigabe werden riskante oder weitreichende Automationen permanent. Korrekturen sind besonders wertvoll und sollten das User Model gezielt aktualisieren.

8.4 Kontrolloberfläche „What JARVIS Knows“

- Liste aller gespeicherten Präferenzen, Routinen und Projekthypothesen.
- Quelle, Confidence, letzte Bestätigung und betroffene Automationen sichtbar.
- Actions: Correct, Forget, Pin, Don't Learn This, Make Temporary.
- Privacy Mode: kein Verhalten lernen, optional kein Gesprächs-Memory, Screen/Kamera-Learning aus.
- „Jarvis, vergiss die letzten 30 Minuten“ erzeugt einen nachvollziehbaren Delete-Workflow.

9. VOICE ENGINE: FLÜSSIG, CLEAN, EIGENE JARVIS-IDENTITÄT

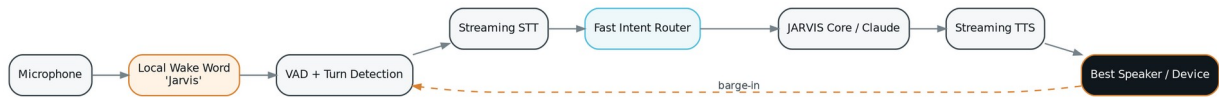


Abbildung 4 - Streaming Voice-Pipeline. Wake, STT, Intent und TTS laufen nicht als serielle Blockkette.

9.1 Voice Personality Contract

- Originale Stimme, nicht Klon einer realen Person. Tiefe/ruhige männliche oder androgyne Tonlage, klare Aussprache, elegant-britische Färbung optional.
- Keine abgehackten Satzstücke; phrase-level streaming, natürliche Prosodie und dynamische Pausen.
- Sarkasmus trocken und selten; schwarzer Humor kontextabhängig; in Notfall, Trauer, Medizin, Security oder ernststen Konflikten Humor automatisch auf 0.
- No-Filler Rule: nicht ständig „Natürlich“, „Sehr gerne“, „Absolut“. Bei simplen Aktionen oft nur „Erledigt.“ oder ein Sound Cue.
- Barge-in: „Jarvis, stopp“ oder neue Frage beendet die laufende Sprachausgabe sofort.
- Night/Whisper/Silent Mode; sensitive Inhalte auf Kopfhörer/Handy statt laut im Raum.
- Sprechtempo und Detailgrad passen sich Aufgabe, Umgebung und deinem erlernten Stil an.

9.2 Latenz-Budget - Zielwerte, keine Garantie

Messpunkt	Ziel
Wake UI/Ack nach Erkennung	< 150-250 ms wahrgenommenes Feedback
Stop/Barge-in	< 150 ms bis Audio stoppt
Local safe action dispatch	< 300 ms bis Start
Fast intent classification	typisch < 100 ms lokal/kleines Modell
Erstes TTS-Audio einfache Antwort	ca. 0.5-1.0 s Ziel unter guten Bedingungen
HUD frame rate	60 FPS minimum target; 120 FPS auf geeigneter Hardware
Long mission status	sichtbares Event spätestens alle 1-3 s, solange Aktivität besteht

9.3 Wake Word auf Geräten

Home Assistant unterstützt 2026 u.a. „Hey Jarvis“ über microWakeWord und auf Android sogar Wake-Word-Erkennung im Hintergrund/bei gesperrtem Gerät; das ist ein sehr guter Prototyping-Pfad für Voice Satellites. [S8] [S9A] Für unsere eigene App bauen wir eine plattformabhängige Wake-Schicht. Android ist für Always-on-Wake realistischer; iOS setzt strengere Background-Audio-/Hotword-Grenzen, daher muss dort Push-to-talk/Siri Shortcut/Action Button als Fallback geplant werden.

10. DEVICE MESH: PC, HANDY, HOME, ALEXA

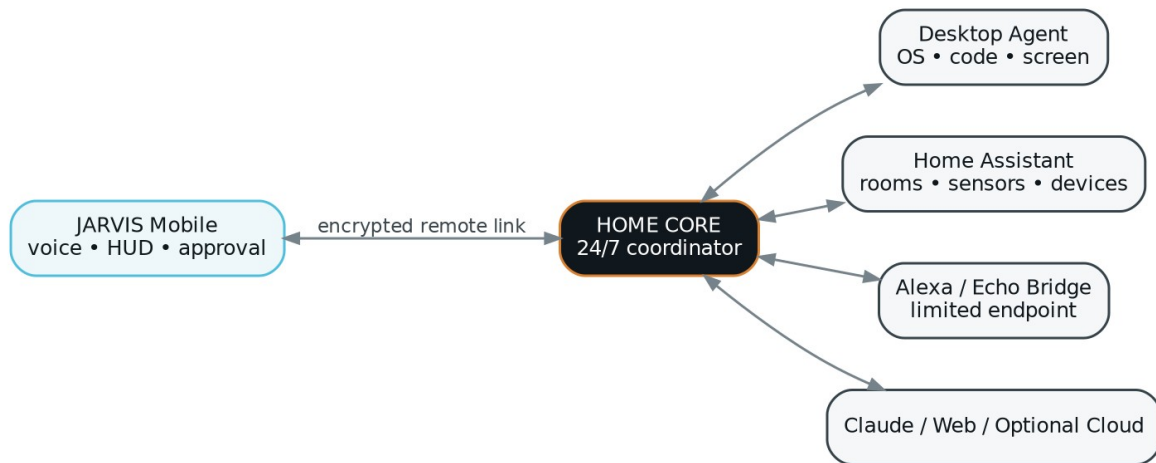


Abbildung 5 - Home Core als privater 24/7-Knoten zwischen Mobile, Desktop, Home Assistant und Cloud.

10.1 Home Core

Ein kleiner x86-Home-Server/Mini-PC läuft 24/7 und hält Mission State, Memory, Device Registry, Event Bus, Home Assistant-Anbindung und Remote Gateway. Dadurch funktioniert Remote-Steuerung auch wenn der Desktop ausgeschaltet ist. Kritische lokale Funktionen bleiben ohne Cloud erreichbar.

10.2 Desktop Agent

- Startet mit dem OS als signierter Hintergrunddienst plus sichtbares HUD.
- OS/App/Window/File/Terminal/Browser-Fähigkeiten hinter Capability Registry.
- Screen capture und optional UI automation; API-first, GUI-automation nur als Fallback.
- Kann WOL-ready sein; meldet „online“ erst nach Heartbeat und Agent-Handshake.
- Workspace restore: Apps, Fenster, Projekt, Terminal und relevante Tabs wiederherstellen.

10.3 Mobile Agent

- Voice, HUD, Push Notifications, Mission Status, approvals, Device Map.
- Remote Befehle werden Ende-zu-Ende authentifiziert; kein offener Home-Port.
- Kamera optional als situativer Sensor; kein heimliches Dauertracking.
- Location nur opt-in und policy-basiert für sinnvolle Automationen.

10.4 Alexa / Echo

PLATTFORMGRENZE Alexa/Echo wird als Bridge/Satellit integriert. Amazon kontrolliert Wake Word, Skill-Umgebung und Display-Fähigkeiten; deshalb darf die Gesamtarchitektur niemals davon abhängen, Alexa vollständig zu übernehmen. Volle JARVIS-UI gibt es auf eigenen Apps/Displays.

10.5 „JARVIS soll sich überall zeigen“

Der Presence Service entscheidet anhand von aktuellem Gerät, Raum, Aktivität, Audio-Routing und Privacy, auf welchen Displays der Core erscheint. Ein globales Wake Event kann synchron an vertrauenswürdige Geräte gesendet werden, aber nur der beste Audio-Satellit antwortet, um Echo-Effekte zu vermeiden.

11. SMART HOME UND DIGITAL TWIN

Home Assistant bleibt Geräte-Gateway. JARVIS spricht nicht direkt 80 verschiedene Herstellerprotokolle, wenn Home Assistant bereits eine stabile Abstraktion bietet.

11.1 Home State Machine

```
HOME STATES
Home / Away / Sleep / Work / Movie / Guests / Night / Vacation

Each state defines:
• allowed proactive notifications
• lighting/climate defaults
• privacy rules
• speaker routing
• device policies
```

11.2 Digital Twin

Später bekommt das HUD einen 2D/3D-Grundriss mit Räumen, Geräten, Temperaturen, Luftqualität, Energie, Türen/Fenstern, Kameras und Missionen. „Warum ist es oben warm?“ kann dann direkt zu Raum-/Sensoranalyse führen.

11.3 Home Safety

- Security-relevante Geräte (Türschloss, Alarm, Garagentor) erhalten höhere Permission Levels als Licht/Musik.
- Kameradaten standardmäßig lokal und ereignisorientiert verarbeiten; klare Retention.
- Keine automatische Entriegelung nur aufgrund einer Sprachaufnahme.
- Power/Heizung/Netzwerk-Aktionen mit sicheren Zustandsmodellen und Fallbacks.

12. LIVE CODING / CREATOR MODE

12.1 Was sichtbar wird

Panel	Inhalt
Project Tree	Dateien, neu/geändert/gelöscht, active file.
Editor	Monaco, live diffs, current patch, syntax.
Terminal	Build-/Test-Ausgabe mit Filter und Error highlights.
Agent Rail	Coordinator, Implementer, Tester, Verifier - Status, keine Chain-of-Thought.
Mission Timeline	Plan, Tasks, Blocker, approvals, checkpoints.
Preview	Website/App/Game direkt starten und sichtbar testen.
Quality	Tests, coverage, lint, performance, visual regression.
Artifact	finaler Build, installer, screenshots, changelog.

12.2 Coding Safety

- Neue Projekte/Skills zuerst in Sandbox oder isoliertem Workspace.
- Branches/Commits pro Feature; keine unkontrollierten Änderungen auf main.
- Tests und Verifier vor „fertig“.
- Dependencies werden bewertet und lockfiles versioniert.
- Builds dürfen Systemressourcen nicht so saturieren, dass HUD/Voice ruckeln.

13. WORLD INTELLIGENCE GLOBE

13.1 Datenpipeline

```
News/Web Sources
-> ingest + timestamp
-> deduplicate / cluster story
-> extract countries/regions/topics
-> source quality + confidence
-> summarize per event
-> geospatial event model
-> JARVIS event bus
-> 3D globe / cards / voice
```

13.2 Interaktionsmodell

- „Was ist heute weltweit wichtig?“ -> Top-Events, Globe dreht in Sprachreihenfolge.
- „Nur KI und Technologie“ -> andere Layer ausblenden.
- Klick auf Land -> Country Brief, Quellen, Timeline, verwandte Ereignisse.
- „Was hat sich seit gestern geändert?“ -> Story Evolution statt Wiederholung.
- „Beweise es“ -> Evidence Mode mit Quellen, Daten und widersprüchlichen Berichten.
- Breaking News wird visuell als vorläufig markiert; Prognosen klar von Fakten trennen.

14. PROAKTIVITÄT, ATTENTION UND BACKGROUND MISSIONS

14.1 JARVIS darf nicht nerven

Proaktivität wird über eine Relevance Engine geregelt. Jedes Event erhält Dringlichkeit, Relevanz, Confidence, Kosten einer Unterbrechung und bevorzugten Kanal. Kritisch -> sofort; wichtig -> opportunistisch; informativ -> Daily Brief; irrelevant -> still.

14.2 Background Missions

Missionen laufen persistent. Wenn ein Agent, PC oder Internet ausfällt, bleibt der Checkpoint erhalten. Nach Recovery kann JARVIS fortsetzen oder dich fragen. Watchdogs erkennen hängen gebliebene Tasks. Budgets begrenzen Zeit, Tokens, Kosten und Tool-Aufrufe.

14.3 Predictive Assistance

JARVIS darf ungefährliche Vorbereitungen antizipieren: benötigte Projektdateien indexieren, Gerätestatus prüfen, Home-Szene vorschlagen. Riskante Aktionen werden nicht „vorausseilend“ ausgeführt. Prediction soll Latenz senken, nicht Kontrolle entziehen.

15. SELBSTLERNEN UND SELBSTERWEITERUNG - KONTROLLIERT

15.1 Skill Factory

User request: "Lerne meinen 3D-Drucker zu bedienen."

Research docs -> design adapter -> generate skill -> tests -> sandbox
 -> security review -> capability manifest -> user approval -> install
 -> versioned skill registry -> monitoring -> rollback available

15.2 Was JARVIS selbst ändern darf

Bereich	Autonomie
Antwortstil / UI-Präferenz	automatisch im sicheren Bereich
Harmloser Shortcut / Vorschlag	automatisch vorschlagen
Routine mit externen Aktionen	nach deiner Freigabe
Neuer Skill	entwickeln + testen autonom; installieren nach Review/Freigabe
Core-Code	Branch/PR erstellen; niemals heimlich in Produktion übernehmen
Security Policy / Kill Switch	niemals selbst lockern oder abschalten
Admin-/Root-Rechte	nur explizit, zeitlich begrenzt, stark bestätigt

16. PERFORMANCE-ARCHITEKTUR: „SUPER FLÜSSIG“ ALS ANFORDERUNG

16.1 Fast Path / Smart Path / Deep Path

```
FAST PATH (milliseconds)
Wake, volume, lights, app open, timer, simple device state
-> local intent rules -> local adapter

SMART PATH (sub-second to seconds)
Ambiguous command, project context, small reasoning
-> fast model / cached context

DEEP PATH (seconds to minutes)
Research, coding, complex mission
-> Opus/Sonnet agents -> tools -> verifier -> streaming status
```

16.2 Anti-Lag-Regeln

- UI renderer, audio capture, wake-word worker und Core-Agenten in getrennten Threads/Prozessen.
- Keine synchrone DB-/Netzwerkoperation im UI thread.
- Heavy jobs bekommen CPU/GPU limits und niedrigere Priorität als Audio/HUD.
- Context caching, project indexing und preloaded tool catalogs.
- Agentenstatus wird event-driven gepusht; kein aggressives UI polling.
- Progressive loading: Globe/3D-Effekte nach Bedarf, Kerninteraktion sofort.
- Performance telemetry lokal: p50/p95 wake latency, first audio, tool dispatch, frame time, mission recovery.

17. DATA MODEL UND INTERNE SCHNITTSTELLEN

17.1 Kernobjekte

Objekt	Wichtige Felder
UserProfile	identity, preferences, privacy policy, personality settings
Device	id, type, capabilities, trust, room, last_seen, key fingerprint
Capability	name, schema, risk level, required permissions, health
Mission	goal, status, priority, budget, owner, context, checkpoints
Task	mission_id, dependencies, assigned_agent, state, retries
AgentRun	provider, model, effort, tools, cost, start/end, outcome
PermissionDecision	action, risk, rule, approval proof, expiration
MemoryItem	type, subject, value, confidence, source, sensitivity, retention
Artifact	file/build/report/preview, hash, path, version
Event	typed envelope, correlation, priority, payload
AuditRecord	immutable summary of sensitive state change

17.2 Capability Manifest

```

name: computer.open_app
version: 1.0
risk: P1
inputs:
  app_id: string
requires:
  - device.trusted
side_effects: true
reversible: false
verifier: computer.process_running
timeout_ms: 10000

```

18. PRIVATE GITHUB, INSTALLER UND EIGENTUM

18.1 Repository-Modell

Der Quellcode liegt in einem privaten GitHub-Repository. GitHub dokumentiert private Repositories als nur für dich bzw. explizit freigegebene Personen zugänglich. [S11] Das persönliche Memory, Secrets, Gespräche, Logs und Home-Daten gehören nicht ins Repository.

```
PRIVATE GITHUB
  source code / tests / docs / CI
    |
    v
  GitHub Actions -> signed build -> private Release
    |
    v
  JARVIS Setup.exe / MSI / DMG / AppImage / mobile build
    |
    v
  LOCAL INSTALL + LOCAL PERSONAL DATA
```

18.2 Release-Modell

GitHub Releases können Binärdateien zu versionierten Releases bündeln; jeder mit Read-Zugriff kann sie sehen. [S12] Für dein Projekt bleiben Releases privat. Später bekommt JARVIS einen eigenen Updater, der signierte private Releases prüft, Backup erstellt, aktualisiert und bei Fehlern zurückrollt.

18.3 Datenpfade

```
C:\Program Files\JARVIS\          -> application binaries
%LOCALAPPDATA%\JARVIS\           -> cache / logs
%APPDATA%\JARVIS\                 -> settings
Encrypted local data volume       -> memory / project index / secrets metadata
Home Core PostgreSQL              -> synchronized durable state
```


19. HARDWARE UND SOFTWARE: WAS DU BRAUCHST

19.1 Entwicklungs-PC - empfohlen

Komponente	Empfehlung
CPU	moderner 8-Core oder besser; Agenten/Builds profitieren von Parallelität
RAM	32 GB empfohlen; 16 GB Minimum für komfortablen Start
SSD	1 TB NVMe empfohlen; Quellcode, Docker, Builds und lokale Modelle wachsen schnell
GPU	nicht zwingend für Cloud-Claude; sinnvoll für lokale Vision/STT/TTS/3D
Mikrofon	sauberes USB/Headset-Mic für Voice-Tests
Netzwerk	Gigabit Ethernet/Wi-Fi 6+; stabiler Router
OS	Windows 11 ist für dein Ziel sehr gut; Architektur bleibt cross-platform

19.2 Home Core

Komponente	Empfehlung
Mini-PC/Home Server	x86, 4-8 Cores, 16-32 GB RAM, 512 GB+ NVMe
Netzwerk	Ethernet, feste DHCP lease
Power	optional UPS für 24/7 Stabilität
Services	JARVIS Core, PostgreSQL, Home Assistant, optional local STT/TTS
Security	separater Device-Key, encrypted disk, automatic backups

19.3 Software installieren

- ☐ Git + GitHub Account; neues privates Repository.
- ☐ Claude Desktop/Claude Code; Max 20x aktiv.
- ☐ Python 3.12 + uv.
- ☐ Node.js 20+ + pnpm.
- ☐ Rust stable toolchain + Tauri prerequisites.
- ☐ Docker / Docker Compose.
- ☐ PostgreSQL + pgvector (zunächst via Docker Compose).
- ☐ VS Code oder bevorzugte IDE.
- ☐ Home Assistant für Home-Phase.
- ☐ Mesh VPN/WireGuard-Lösung für Remote-Phase.
- ☐ Optional Android Testgerät als frühes „Hey Jarvis“-Voice-Satellite.

20. REPOSITORY-STRUKTUR

```

jarvis/
├── apps/
│   ├── desktop/           # Tauri + React HUD
│   ├── mobile/           # Tauri/mobile shared UI + native plugins
│   └── admin/            # optional diagnostics
├── core/
│   ├── api/
│   ├── state/
│   ├── events/
│   ├── missions/
│   ├── permissions/
│   ├── memory/
│   ├── models/
│   ├── agents/
│   ├── capabilities/
│   ├── verifier/
│   └── scheduler/
├── adapters/
│   ├── desktop/
│   ├── browser/
│   ├── home_assistant/
│   ├── news/
│   ├── github/
│   └── alexa_bridge/
├── voice/
│   ├── wake/
│   ├── stt/
│   ├── tts/
│   └── routing/
├── skills/
├── mcp/
├── packages/
│   ├── protocol/
│   ├── ui-kit/
│   └── skill-sdk/
├── infra/
│   ├── docker/
│   ├── home-core/
│   └── ci/
├── docs/
│   ├── SPEC.md
│   ├── SECURITY.md
│   ├── PERFORMANCE.md
│   └── decisions/ADR-0001...
├── tests/
└── CLAUDE.md
  
```

21. BUILD-REIHENFOLGE - DER WICHTIGSTE TEIL



Abbildung 6 - Reihenfolge: Gehirn und Sicherheitsgrenzen vor Voice/Design und fortgeschrittenen Features.

NICHT MIT DEM HUD ANFANGEN Der schnellste Weg zu einem beeindruckenden, aber unbrauchbaren Projekt wäre zuerst 3D-Ringe, Globus und Voice zu bauen. Wir bauen zunächst die unsichtbare Systemwahrheit: State, Missionen, Events, Permissions, Tools, Verifikation und Memory.

PHASE 0 - Foundation / Source of Truth

Repository, Toolchain, Specs, CI und Architekturentscheidungen.

Schritte

1. Privates GitHub-Repo anlegen; `main` schützen; Branch-Workflow festlegen.
2. Diese PDF als `docs/JARVIS\_Master\_Blueprint\_1.0.pdf` ablegen und daraus `SPEC.md` erzeugen.
3. `CLAUDE.md` mit Projektgesetzen, Build/Test-Kommandos und Security-Regeln erstellen.
4. Docker Compose mit PostgreSQL/pgvector; `.env.example`, keine Secrets committen.
5. CI: format, lint, unit tests, secret scan, build smoke test.

Exit-Kriterien

- ☐ Clean clone -> one setup command -> tests green.
- ☐ Keine Secrets im Repo.
- ☐ Architecture Decision Records vorhanden.

PHASE 1 - Brain Core 0.1

State, Event Bus, Mission Engine, API und Mock-Capabilities.

Schritte

6. Core process + health endpoint.
7. Typed Event Bus und durable event/task persistence.
8. Mission state machine.
9. Capability Registry mit mock `echo`, `clock`, `open\_url`.
10. Minimaler WebSocket event stream fürs Debug UI.

Exit-Kriterien

- ☐ Mission überlebt Restart.
- ☐ Events sind korreliert und replaybar.
- ☐ Mock-Tool wird nur nach Permission ausgeführt.

PHASE 2 - Permission + Execution + Verification

Sicherer Gatekeeper zwischen KI und Welt.

Schritte

11. P0-P6 risk classes im Code.
12. Policy evaluator + approval workflow.
13. Execution Gateway + timeout/retry.
14. Verifier interface.
15. Immutable/audit-oriented log.
16. Kill switch.

Exit-Kriterien

- ☐ Risky mock action wartet auf Approval.
- ☐ Denied action kann vom Modell nicht umgehen werden.
- ☐ Verifier kann Tool-Erfolg als Ziel-Fehler markieren.

PHASE 3 - Claude Brain / Agent Runtime

Claude kontrolliert planen, ohne Direktzugriff auf Systeme.

Schritte

17. `IntelligenceProvider` Interface.
18. Claude Agent SDK adapter.
19. Opus 5/Sonnet 5 model routing.
20. Agent budgets: time/token/cost/tools.
21. Subagent orchestration für Research/Code/Test/Verify.
22. Streaming agent events in Event Bus.

Exit-Kriterien

- ☐ Agent kann komplexe Mock-Mission abschließen.
- ☐ Tool allowlist wird eingehalten.
- ☐ Abbruch/Timeout/Retry funktionieren.

PHASE 4 - Memory + User Model

Langzeitkontext und kontrolliertes Lernen.

Schritte

23. Memory schema + vector retrieval.
24. Working/Episodic/Semantic/Project/Preference stores.
25. Memory writer mit sensitivity/retention.
26. Context Builder.
27. Correction feedback loop.
28. "What JARVIS Knows" API.

Exit-Kriterien

- ☐ Explizit gespeicherte Fakten werden später korrekt retrieved.
- ☐ Vergessen löscht/entkoppelt Daten reproduzierbar.
- ☐ Confidence/Source ist sichtbar.

PHASE 5 - Voice 0.1

Wake -> STT -> Core -> TTS mit Unterbrechung.

Schritte

29. Lokales wake-word worker.
30. VAD/turn detection.
31. Streaming STT adapter.
32. Streaming TTS adapter.
33. Barge-in/stop.
34. Voice personality contract.
35. Latency telemetry.

Exit-Kriterien

- ☐ Wake acknowledgement fühlt sich sofort an.

- ☐ Barge-in stoppt Audio schnell.
- ☐ Simple local command braucht kein Claude.

PHASE 6 - Desktop Agent + Minimal HUD

JARVIS wird auf dem PC real.

Schritte

36. Tauri/React shell.
37. Presence states.
38. Desktop service: apps/processes/files/window state.
39. Screen capture adapter.
40. Safe terminal/browser capabilities.
41. Approval UI.
42. Event rail.

Exit-Kriterien

- ☐ App öffnen und verifizieren.
- ☐ Screen Context optional an Core.
- ☐ HUD bleibt flüssig während Agent arbeitet.

PHASE 7 - Live Coding Mode

Claude Code/Agent SDK wird sichtbarer Creator.

Schritte

43. Monaco Editor + file tree.
44. Diff/event streaming.
45. Terminal build/test panels.
46. Sandboxed project runner.
47. Preview window.
48. Writer/Test/Verifier pattern.
49. Artifact registry.

Exit-Kriterien

- ☐ Game/Web-Prototyp kann erstellt, gebaut, gestartet und verifiziert werden.
- ☐ Fehler wird im HUD sichtbar und repariert.

PHASE 8 - Home Core + Home Assistant

24/7-Hub und Haussteuerung.

Schritte

50. Core auf Mini-PC deployen.
51. Home Assistant adapter.
52. Device registry + room graph.
53. Voice satellite integration.
54. WOL service.
55. Home states/scenes.
56. Local offline paths.

Exit-Kriterien

- ☐ Handy/PC kann Licht/Testdevice über Core steuern.
- ☐ Desktop kann über WOL geweckt werden (Hardware vorausgesetzt).

- ☐ Cloud-Ausfall zerstört Home-Basics nicht.

PHASE 9 - Mobile + Secure Remote

JARVIS unterwegs.

Schritte

57. Mobile HUD/voice.
58. Device enrollment + keypairs.
59. Mesh VPN/secure tunnel.
60. Push notifications.
61. Biometric/passkey approvals.
62. Mission handover desktop<->mobile.

Exit-Kriterien

- ☐ Unterwegs kann sichere Testaktion zuhause ausgeführt werden.
- ☐ Gestohlenes Gerät kann revoked werden.
- ☐ Mission bleibt geräteübergreifend identisch.

PHASE 10 - Dynamic HUD + World Globe

Volle visuelle Identität.

Schritte

63. GPU scene manager.
64. AI Core animation states.
65. 3D Globe + country selection.
66. News event data model.
67. Adaptive panels/transitions.
68. Performance quality tiers.

Exit-Kriterien

- ☐ News drehen korrekt zu Land/Region.
- ☐ Globe 60 FPS target unter typischer Last.
- ☐ UI zeigt echte Events, keine erfundenen States.

PHASE 11 - Proactivity + Learning

JARVIS beginnt wirklich „mitzudenken“.

Schritte

69. Relevance engine.
70. Habit detector.
71. Automation suggestions.
72. Background missions/watchdogs.
73. Daily brief.
74. Predictive preloading.
75. Privacy/guest modes.

Exit-Kriterien

- ☐ Irrelevante Events bleiben still.
- ☐ Neue Routine wird vorgeschlagen, nicht blind aktiviert.
- ☐ Background Mission überlebt Service-Restart.

PHASE 12 - Skill Factory + Release 1.0

Sichere Erweiterbarkeit und installierbares Produkt.

Schritte

76. Skill SDK + manifest.
77. Skill sandbox/test/review/install.
78. Private signed release pipeline.
79. Installer + updater + rollback.
80. Encrypted backup/restore.
81. Full security/performance regression suite.

Exit-Kriterien

- ☐ Clean machine install.
- ☐ Update + rollback getestet.
- ☐ Skill kann erstellt werden, ohne Core-Security zu umgehen.
- ☐ 1.0 acceptance suite komplett grün.

22. DIE ERSTEN 10 COMMITS - KONKRETER START

#	Commit	Resultat
001	chore: bootstrap monorepo	Repo, README, license/private note, `.gitignore`, tooling.
002	docs: add master spec and ADR template	PDF, SPEC.md, SECURITY.md, PERFORMANCE.md.
003	infra: postgres compose	PostgreSQL/pgvector + migrations skeleton.
004	core: typed event bus	Event envelope, in-process pub/sub, persistence.
005	core: mission state machine	Mission/Task tables + transition tests.
006	security: permission engine	P0-P6, deny/ask/allow, unit tests.
007	core: capability registry	Manifest, mock tools, execution gateway.
008	core: verifier interface	Outcome checks, retry policy.
009	api: websocket event stream	Debug client can watch real events.
010	ai: claude provider skeleton	Agent SDK adapter behind interface, no real OS tools yet.

23. WIE DU CLAUDE MAX 20x / CLAUDE CODE RICHTIG EINSETZT

23.1 Arbeitsregel

Niemals „Baue mir JARVIS komplett“ in einem Durchgang. Jede Session bekommt genau einen abgegrenzten Milestone mit Definition of Done. Claude darf Dateien verändern, Tests ausführen und Diff reviews - aber immer innerhalb des Branches und der aktuellen Capability.

23.2 `CLAUDE.md` Kernregeln

```
# JARVIS DEVELOPMENT LAWS
1. Read docs/SPEC.md, SECURITY.md, PERFORMANCE.md before architectural changes.
2. Never bypass PermissionEngine or ExecutionGateway.
3. No model/provider-specific logic outside adapters/models.
4. Every side-effecting capability has risk level + verifier.
5. Write/update tests with every behavior change.
6. No secrets in code, prompts, logs, fixtures or commits.
7. Keep UI/audio threads non-blocking.
8. Do not claim a task succeeded before verifier passes.
9. Core security policy may only become stricter without explicit owner approval.
10. If spec and code conflict, stop and report the conflict.
```

23.3 Master Prompt für Architektur-Sessions

You are the principal architect for Project JARVIS.
Read docs/JARVIS_Master_Blueprint_1.0.pdf (or SPEC.md), SECURITY.md and existing ADRs.
Use the highest available Claude reasoning model and maximum appropriate effort.

Task: <ONE CLEAR MILESTONE>

Before coding:

1. Map the affected modules and invariants.
2. Identify security and performance risks.
3. Propose an implementation plan and acceptance tests.
4. Do not modify files until the plan is coherent.

During coding:

- make minimal, modular changes
- run tests/lint/type checks
- emit no placeholders for required behavior
- preserve rollback path

After coding:

- review the diff as a skeptical maintainer
- run acceptance tests
- list remaining risks
- do not say "done" unless tests and verifier criteria pass.

23.4 Writer-Verifier Workflow

- Agent A: implementiert den Milestone.
- Agent B: erhält Spec + Diff, sucht Bugs, Sicherheitsumgehungen, Race Conditions und fehlende Tests.
- Agent C nur bei kritischen Modulen: Security/Performance Review.
- Coordinator akzeptiert Änderungen erst nach grünem Testset und abgearbeiteten Findings.

24. TEST- UND QUALITÄTSSTRATEGIE

Testart	Was geprüft wird
Unit	State transitions, policies, parsers, memory scoring, routing.
Contract	Capability schemas, agent events, device protocol, API compatibility.
Integration	Core + DB + Agent SDK + adapters.
Simulation	Fake smart home / fake PC / fake news feeds für deterministische Tests.
Security	Prompt injection, permission bypass, secret leakage, forged device events.
Chaos	Service restart, lost network, delayed tool, corrupted cache, agent timeout.
Voice	Wake false positives/negatives, barge-in, noisy room, latency.
UI	60-FPS budget, long lists, globe stress, layout regression.
E2E	„Wake PC -> open project -> build -> verify -> notify mobile“.
Recovery	Backup restore, updater rollback, mission resume after crash.

24.1 Golden Scenarios für Release 1.0

- ☐ „Jarvis“ -> sauberes Voice-HUD -> simple local action unter Fast Path.
- ☐ Unterwegs -> PC wake -> app open -> verified result -> mobile notification.
- ☐ Coding mission -> files/diffs/tests visible -> app/game starts -> verifier passes.
- ☐ World news -> story clusters -> globe rotates -> country/source evidence correct.
- ☐ Home scene -> devices change -> JARVIS verifies states.
- ☐ Risky command -> biometric approval -> action -> audit + undo where possible.
- ☐ Internet down -> local wake/home/basic PC functions remain available.
- ☐ Core restart -> active mission recovers from checkpoint.
- ☐ Memory correction -> old assumption updated and source/confidence visible.
- ☐ Kill switch -> all side-effecting agent work stops immediately.

25. RISIKOREGISTER UND ENTSCHEIDUNGEN

Risiko	Warum wichtig	Strategie
Zu früh zu viel UI	verbrannt Zeit, Core bleibt instabil	Full HUD erst nach Core/Permissions/Memory; minimal debug UI früh.
Claude als monolithisches Gehirn	Vendor lock-in, Kosten, Offline tot	Provider abstraction + local fast path.
Voice latency	zerstört JARVIS-Gefühl	streaming, local wake, local intents, warm services.
Falsche Autonomie	gefährliche Aktionen	risk levels, approvals, verifier, audit, undo.
Memory wird Müllhalde	schlechter Kontext und Privacy-Risiko	typed memory, confidence, decay, source, retention.
Multi-Agent-Overkill	Kosten/Latenz/Fehler	nur klare parallelisierbare Tasks; Writer-Verifier standard.
Alexa-Limits	falsche Erwartung	Bridge statt Core; eigene Displays für volle UI.
iOS Wake-Limits	Always-on eventuell nicht möglich	Android zuerst; Push-to-talk/OS shortcuts als iOS fallback.
Self-modification	Core destabilisiert/sicherheitskritisch	PR/branch + tests + owner approval; policy immutable.
Cloud-Ausfall	System unbrauchbar	local home/control/voice fallbacks, persistent state.

26. 30-TAGE-STARTPLAN (FOKUS AUF RICHTIGE REIHENFOLGE)

Dieser Zeitplan ist ein Arbeitsrahmen, keine Garantie. Mit Claude Code kann vieles schneller gehen; Security und Testing werden nicht übersprungen.

Zeitraum	Fokus	Ergebnis
Tage 1-2	Foundation	Repo, toolchain, specs, CI, DB.
Tage 3-7	Brain Core	Events, Missions, Capability Registry, debug stream.
Tage 8-10	Security	Permissions, execution gateway, verifier, kill switch.
Tage 11-14	Claude integration	Agent SDK adapter, model router, budgets, mock mission.
Tage 15-19	Memory	structured stores, retrieval, user model, forget/correct.
Tage 20-24	Voice prototype	wake, STT, TTS, barge-in, latency metrics.
Tage 25-30	Desktop shell	Tauri HUD shell, events, basic PC capability, approval UI.

27. 200+ OPTIMIERUNGEN / ADVANCED FEATURE BACKLOG

Diese Liste ist bewusst größer als Release 1.0. Sie verhindert, dass wir spätere Fähigkeiten architektonisch blockieren. Features werden nach Core-Reife priorisiert, nicht blind nacheinander eingebaut.

- 001 Instant-Wake-Pipeline: Das Wake Word läuft vollständig lokal und bekommt höchste Prozesspriorität.
- 002 Warm Brain: Häufig benötigte Modelle und Dienste bleiben vorgeladen, damit nichts erst starten muss.
- 003 Predictive Preloading: JARVIS lädt wahrscheinliche nächste Tools schon im Hintergrund.
- 004 Adaptive Model Routing: Kleine Aufgaben gehen an schnelle Modelle, komplexe an Claude.
- 005 Parallel Tool Execution: Unabhängige Aufgaben laufen gleichzeitig statt nacheinander.
- 006 Streaming Everywhere: Sprache, Text, Tool-Ergebnisse und UI-Updates werden kontinuierlich gestreamt.
- 007 Sub-100-ms Feedback: Jeder Befehl bekommt sofort sichtbares oder akustisches Feedback.
- 008 GPU-priorisiertes HUD: KI-Berechnungen dürfen niemals die UI-Animationen blockieren.
- 009 Background Compute Scheduler: Große Jobs laufen bevorzugt dann, wenn dein Rechner wenig ausgelastet ist.
- 010 Battery-Aware Mode: Auf Handy und Laptop reduziert JARVIS automatisch Rechenlast bei niedrigem Akku.
- 011 Thermal Awareness: Bei hoher CPU/GPU-Temperatur reduziert er Hintergrundarbeit.
- 012 Network-Aware Routing: Schlechte Internetverbindung -> mehr lokale Verarbeitung.
- 013 Adaptive Quality: 3D-Effekte skalieren automatisch je nach Gerät.
- 014 Zero-Reload UI: Das HUD bleibt dauerhaft aktiv; keine klassischen Seiten-Reloads.
- 015 Persistent Sessions: Neustart des PCs verliert keine laufenden Missionen.
- 016 Hot Module Reloading: Einzelne JARVIS-Dienste können aktualisiert werden, ohne alles neu zu starten.
- 017 Priority Queues: Notfall- und direkte User-Befehle überholen Hintergrundjobs sofort.
- 018 Latency Profiler: JARVIS misst selbst, welcher Teil einer Antwort langsam war.
- 019 Automatic Performance Tuning: Er optimiert Cachegrößen und Worker-Zahl anhand deiner Hardware.
- 020 Performance History: Du kannst fragen: „Warum warst du gestern Abend langsam?“
- 021 True Duplex Voice: Du und JARVIS könnt theoretisch gleichzeitig sprechen, ohne dass alles zusammenbricht.
- 022 Smart Barge-In: Er erkennt, ob du ihn wirklich unterbrechen willst oder nur „mhm“ sagst.
- 023 Adaptive Speaking Speed: Technische Erklärungen langsamer, einfache Statusmeldungen schneller.
- 024 Dynamic Prosody: Sarkasmus, Warnung, Erfolg und normale Antworten klingen unterschiedlich.
- 025 Room-Aware Volume: Lautstärke passt sich Raum und Umgebungsgeräuschen an.
- 026 Distance-Aware Voice: Bist du weiter entfernt, spricht er etwas kräftiger.
- 027 Night Voice: Nachts automatisch ruhiger und weniger basslastig.
- 028 Headphone Privacy: Sensible Antworten bevorzugt nur über deine Kopfhörer.
- 029 Speaker Selection: Er entscheidet, welcher Lautsprecher gerade sinnvoll ist.
- 030 Conversation Handover: Gespräch vom Handy nahtlos auf den PC übernehmen.
- 031 No-Repeat Memory: Er wiederholt keine Erklärung, die du gerade schon verstanden hast.
- 032 Interruption Recovery: Nach einer Unterbrechung weiß er exakt, wo das Gespräch war.
- 033 Sentence Completion Detection: Er wartet bei natürlichen Denkpausen länger.
- 034 Filler Recognition: „Ähm“, „warte“, „also“ wird nicht als Befehl interpretiert.
- 035 Conversational Compression: Nach längerem Gespräch hält er relevante Punkte kompakt im Arbeitskontext.
- 036 Personal Vocabulary: Eigene Namen, Spitznamen, Projekte und Fachbegriffe werden zuverlässig verstanden.
- 037 Pronunciation Memory: Du kannst ihm beibringen, wie bestimmte Namen ausgesprochen werden.
- 038 Language Switching: Deutsch -> Englisch mitten im Satz ohne Neustart des Gesprächs.
- 039 Whisper Detection: Flüsterst du, antwortet er ebenfalls leiser.
- 040 Nonverbal Feedback: Kurzer Ton statt Satz, wenn Worte überflüssig wären.
- 041 Attention Model: JARVIS unterscheidet zwischen gerade wichtig und nur irgendwann interessant.
- 042 Intent Confidence: Jeder erkannte Befehl bekommt intern eine Sicherheitseinschätzung.
- 043 Ambiguity Detector: Bei riskanten Mehrdeutigkeiten fragt er nach, bei harmlosen entscheidet er selbst.
- 044 Current Focus Model: Er weiß, woran du aktuell arbeitest.
- 045 Goal Hierarchy: Tagesziele, Projektziele und langfristige Ziele werden getrennt verwaltet.

- 046 Decision Memory: Nicht nur was entschieden wurde, sondern warum.
- 047 Contradiction Detection: Neue Informationen widersprechen altem Memory -> JARVIS weist darauf hin.
- 048 Memory Decay: Unwichtige Informationen verlieren mit der Zeit Gewicht.
- 049 Memory Reinforcement: Häufig bestätigte Fakten werden stabiler.
- 050 Temporary Memory: „Nur bis morgen merken“ wird automatisch gelöscht.
- 051 Conditional Memory: Informationen nur in einem bestimmten Projektkontext verwenden.
- 052 Private Memory Vault: Besonders sensible Erinnerungen getrennt verschlüsseln.
- 053 Memory Branches: Arbeits- und Privatkontext können getrennte Wissensräume haben.
- 054 Relationship Graph: Er kennt Beziehungen zwischen Personen, Projekten und Firmen.
- 055 Object Memory: Er kann sich mit Zustimmung merken, wo bestimmte Dinge normalerweise liegen.
- 056 Habit Evolution: Er erkennt, wenn sich eine alte Routine verändert.
- 057 Preference Conflict Detection: „Früher mochtest du X, zuletzt nutzt du Y häufiger.“
- 058 Memory Explanations: Auf Nachfrage zeigt er, woher eine Annahme stammt.
- 059 Memory Health Check: Veraltete oder widersprüchliche Erinnerungen regelmäßig überprüfen.
- 060 Personal Digital Biography: Optional eine chronologische, durchsuchbare Geschichte deiner Projekte und wichtigen Ereignisse.
- 061 Mission Planner: Große Ziele werden automatisch in Teilziele zerlegt.
- 062 Mission Dependencies: JARVIS versteht, welcher Schritt auf welchen warten muss.
- 063 Agent Marketplace intern: Spezialagenten dynamisch je nach Aufgabe laden.
- 064 Ephemeral Agents: Für eine einmalige Aufgabe entsteht ein Agent und wird danach wieder gelöscht.
- 065 Long-Lived Agents: Bestimmte Projektagenten behalten dauerhaft Kontext.
- 066 Agent Skill Scores: JARVIS lernt, welcher Agent bei welcher Aufgabe gut funktioniert.
- 067 Agent Competition: Zwei Agenten können unabhängig Lösungen entwickeln; JARVIS wählt die bessere.
- 068 Agent Debate: Spezialisten diskutieren schwierige technische Entscheidungen.
- 069 Agent Supervisor: Ein eigener Supervisor überwacht Agenten auf Fehler und Endlosschleifen.
- 070 Time Budget: Missionen können ein maximales Zeitbudget erhalten.
- 071 Cost Budget: Cloud-Aufgaben bekommen ein Kostenlimit.
- 072 Compute Budget: Ressourcenintensive Aufgaben dürfen nur bestimmte CPU/GPU-Limits verwenden.
- 073 Fallback Plans: Fällt Plan A aus, wird automatisch Plan B versucht.
- 074 Recovery Checkpoints: Lange Missionen speichern Zwischenstände.
- 075 Task Migration: Eine Mission kann von deinem Desktop auf den Home-Server wechseln.
- 076 Scheduled Continuation: „Mach morgen weiter“ setzt die Mission später fort.
- 077 Autonomous Research Queue: Offene Fragen können nachts recherchiert werden.
- 078 Mission Watchdog: Hängt eine Aufgabe fest, erkennt JARVIS das selbst.
- 079 Escalation Logic: Nach mehreren Fehlschlägen fragt er dich statt weiter herumzuprobieren.
- 080 Outcome Scoring: Nach Missionen bewertet er, wie erfolgreich das Ergebnis wirklich war.
- 081 Procedural HUD: UI wird passend zum Prozess aufgebaut, nicht aus statischen Seiten.
- 082 Spatial Panels: Informationen können im 3D-Raum angeordnet werden.
- 083 Focus Zoom: Das relevante Panel fährt automatisch nach vorne.
- 084 Context Collapse: Unwichtige Daten ziehen sich visuell zurück.
- 085 Globe Layers: Nachrichten, Wetter, Wirtschaft und Ereignisse als separat schaltbare Schichten.
- 086 Real-Time Earth Lighting: 3D-Erde zeigt echte Tag-/Nachtseite.
- 087 Live Flight Layer: Optional Flugbewegungen auf dem Globus anzeigen.
- 088 Satellite Layer: Satelliten und Raumfahrtmissionen visualisieren.
- 089 Event Pulses: Große Weltereignisse erzeugen dezente geografische Pulse.
- 090 Historical Globe: Weltgeschehen über eine Timeline zurückspulen.
- 091 AR-Ready HUD: Architektur später für AR-Brillen vorbereiten.
- 092 Gesture Navigation: Auf unterstützten Geräten HUD mit Handgesten bedienen.
- 093 Eye Focus: Später optional erkennen, welches Panel du ansiehst.

- 094 Voice + Gesture Kombination: „Öffne das“ während du auf ein Element zeigst.
- 095 Physical Screen Mapping: JARVIS weiß, welcher Monitor wo auf deinem Schreibtisch steht.
- 096 Multi-Monitor Choreography: Code links, Preview Mitte, Logs rechts automatisch verteilen.
- 097 Ambient Mode: Wenn nichts passiert, wird JARVIS zu einer minimalistischen Umgebung.
- 098 Cinematic Transitions: Wechsel zwischen News-, Coding- und Home-Modus wirkt wie ein Systemwechsel.
- 099 Information Density Dial: Von ultraminimalistisch bis vollständiges Command Center.
- 100 HUD Replay: Nach einer Mission kannst du die wichtigsten Ereignisse visuell erneut ansehen.
- 101 Device Presence Map: Alle deine Geräte mit Online-/Offline-Status in einer Karte.
- 102 Capability Discovery: Neues Gerät im Netz -> JARVIS erkennt, was es kann.
- 103 Device Nicknames: „Der kleine Bildschirm“, „Gaming-PC“, „Wohnzimmer-TV“.
- 104 Smart Routing: „Zeig mir das“ -> automatisch auf dem besten Display.
- 105 Wake-on-LAN Manager: PC, NAS oder andere kompatible Geräte aufwecken.
- 106 Remote Shutdown Guard: Vor Fernabschaltung prüfen, ob wichtige Jobs laufen.
- 107 Remote Desktop Assistance: JARVIS kann dir zeigen, was auf deinem PC passiert.
- 108 Home State Machine: Zuhause kennt Zustände wie Away, Home, Sleep, Movie, Work.
- 109 Arrival Automation: Heimkehr kann Geräte vorbereiten.
- 110 Departure Check: Beim Verlassen prüfen: Fenster, Licht, Geräte, offene Tasks.
- 111 Room Context: JARVIS weiß, in welchem Raum ein Befehl gegeben wurde.
- 112 Room Handover: Stimme und UI folgen dir zwischen Räumen.
- 113 Speaker Mesh: Mehrere Lautsprecher wirken wie eine gemeinsame JARVIS-Stimme.
- 114 Visual Presence Mesh: Animation kann auf mehreren Displays synchron erscheinen.
- 115 Home Digital Twin: Interaktives Modell deines Hauses mit Gerätezuständen.
- 116 Energy Map: Live-Energieverbrauch pro Raum/Gerät.
- 117 Environmental Intelligence: Temperatur, Luftfeuchtigkeit, CO2 und Luftqualität zusammenführen.
- 118 Smart Scene Generator: JARVIS erstellt aus deinen Gewohnheiten neue Home-Szenen.
- 119 Device Health: Batteriestände, Firmware, Verbindungsqualität und Fehler überwachen.
- 120 Offline Home Control: Wichtige Hausfunktionen funktionieren auch ohne Internet.
- 121 Live Code Streaming: Du siehst Dateien nahezu in Echtzeit entstehen.
- 122 Semantic Code Map: Visuelle Übersicht, wie Module miteinander verbunden sind.
- 123 Architecture View: JARVIS baut automatisch Diagramme der Softwarearchitektur.
- 124 Live Build Graph: Build-Schritte werden als Fortschrittsnetz visualisiert.
- 125 Error Heatmap: Bereiche mit vielen Fehlern werden im Projekt hervorgehoben.
- 126 Automatic Test Generation: Zu neuen Funktionen werden direkt Tests erzeugt.
- 127 Visual Regression Tests: JARVIS erkennt, wenn UI unbeabsichtigt anders aussieht.
- 128 Performance Regression Tests: Neue Version langsamer? JARVIS merkt es.
- 129 Dependency Risk Analyzer: Veraltete oder problematische Bibliotheken erkennen.
- 130 Project Sandbox: Generierte Programme zuerst isoliert ausführen.
- 131 Instant Preview: Web-, App- oder Game-Preview direkt im HUD.
- 132 Game Telemetry: Beim Testspiel sieht JARVIS FPS, Fehler, Kollisionen und Logs.
- 133 AI Playtester: Ein Agent spielt dein Game selbst und sucht Probleme.
- 134 UX Test Agent: Ein anderer Agent prüft Bedienbarkeit.
- 135 Automatic Changelog: Jede relevante Änderung wird verständlich dokumentiert.
- 136 Code Time Machine: Projekt auf jeden früheren Zustand zurücksetzen.
- 137 Feature Branch Automation: JARVIS erstellt für Experimente automatisch getrennte Branches.
- 138 Prototype Mode: „Bau erst einen schnellen Prototyp“ nutzt andere Qualitätsregeln als Production.
- 139 Production Mode: Stärkere Tests, Reviews und Security Checks.
- 140 Visual Creator Pipeline: Bilder, UI, Sounds, Code und Content können in einer Mission zusammengebaut werden.
- 141 Global News Radar: Relevante Ereignisse weltweit automatisch clustern.

- 142 Duplicate News Collapse: Viele Artikel über dasselbe Ereignis werden zu einem Ereignis zusammengefasst.
- 143 Source Reliability Model: Unterschiedliche Quellen bekommen transparent unterschiedliche Vertrauenswerte.
- 144 Bias Comparison: Unterschiedliche Darstellungen desselben Ereignisses gegenüberstellen.
- 145 Breaking-News Confidence: Frühmeldungen klar von bestätigten Fakten unterscheiden.
- 146 Story Evolution: „Was hat sich seit gestern geändert?“ statt alles neu erzählen.
- 147 Country Brief: Auf dem Globus Land auswählen -> kompakte Lageübersicht.
- 148 Topic Watch: Bestimmte Themen dauerhaft beobachten.
- 149 Company Intelligence: Firmen, Produkte und Konkurrenten als eigene Wissensobjekte.
- 150 Technology Radar: Neue KI-, Hardware- und Softwareentwicklungen verfolgen.
- 151 Patent Watch: Später relevante öffentliche Patente zu Projektthemen beobachten.
- 152 Research Notebook: Jede größere Recherche bekommt Quellen, Schlussfolgerungen und offene Fragen.
- 153 Fact Conflict View: Widersprüchliche Quellen werden sichtbar nebeneinander gestellt.
- 154 Evidence Chains: JARVIS kann zeigen, welche Quellen eine Aussage stützen.
- 155 Research Freshness: Alte Informationen werden als potenziell veraltet markiert.
- 156 Automatic Recheck: Wichtige Fakten nach einer gewissen Zeit erneut überprüfen.
- 157 Deep Dive Mode: Aus einer Nachricht automatisch technische Hintergründe analysieren.
- 158 Explain Like Expert: Gleicher Sachverhalt auf Anfänger-, Fach- oder Expertenniveau.
- 159 Forecast Separation: Vorhersagen optisch klar von Fakten trennen.
- 160 World Timeline: Große globale Entwicklungen über Wochen oder Monate zusammenhängend darstellen.
- 161 Zero-Trust Tools: Kein Agent bekommt automatisch Zugriff auf alle Werkzeuge.
- 162 Per-Task Permissions: Rechte gelten nur für die aktuelle Mission.
- 163 Expiring Permissions: Temporäre Freigaben laufen automatisch ab.
- 164 Hardware Security Key Support: Sehr kritische Aktionen optional nur mit physischem Schlüssel.
- 165 Passkey Approval: Handy kann kritische Aktionen per Passkey bestätigen.
- 166 Biometric Confirmation: Mobile Freigaben über Face ID/Fingerprint.
- 167 Transaction Preview: Vor größeren Aktionen exakt anzeigen, was verändert wird.
- 168 Immutable Audit Log: Bestimmte Sicherheitslogs können nicht vom Agenten verändert werden.
- 169 Secret Isolation: API-Keys werden niemals direkt in Modellkontexte gegeben, wenn vermeidbar.
- 170 Credential Broker: JARVIS führt authentifizierte Aktionen aus, ohne dem Agenten das Passwort zu zeigen.
- 171 File Quarantine: Unbekannte Downloads zuerst isoliert prüfen.
- 172 Command Allowlist: Besonders gefährliche Shell-Befehle benötigen strengere Regeln.
- 173 Network Segmentation: Smart-Home-Geräte können in getrennten Netzbereichen bleiben.
- 174 Behavior Anomaly Detection: JARVIS bemerkt ungewöhnliche eigene Tool-Nutzung.
- 175 Agent Kill Switch: Ein Befehl stoppt sofort alle Agenten.
- 176 Safe Boot: Bei beschädigter Konfiguration startet nur ein minimaler JARVIS.
- 177 Automatic Backups: Memory, Einstellungen und Projekte werden versioniert gesichert.
- 178 Encrypted Backup: Backups sind ohne deinen Schlüssel unbrauchbar.
- 179 Disaster Recovery: Neuer PC -> JARVIS kann kontrolliert wiederhergestellt werden.
- 180 Security Dashboard: Du siehst aktive Rechte, Geräte, Sessions und letzte kritische Aktionen.
- 181 Personality Continuity: Egal welches Modell benutzt wird, JARVIS bleibt charakterlich derselbe.
- 182 Contextual Sarcasm: Sarkasmus nur dann, wenn die Situation dafür passt.
- 183 Humor Calibration: Er lernt, welche Witze du lustig findest und welche nerven.
- 184 Deadpan Mode: Extrem trockener Humor als eigene Intensitätsstufe.
- 185 Professional Override: In Meetings automatisch weniger Humor.
- 186 Guest Mode: Wenn andere Personen mit JARVIS sprechen, gelten andere Rechte und weniger persönlicher Kontext.
- 187 Owner Priority: Deine Befehle haben gegenüber Gästen höhere Priorität.
- 188 Personal Rituals: Eigene Begrüßungen oder Routinen für bestimmte Situationen entwickeln.
- 189 Milestone Awareness: JARVIS erkennt Projekt-Meilensteine und kann sie hervorheben.

- 190 Achievement Archive: Große abgeschlossene Projekte bekommen eine visuelle Chronik.
- 191 Idea Incubator: Du wirfst spontane Ideen hinein; JARVIS strukturiert sie später automatisch.
- 192 Dream Queue: Verrückte Ideen können langfristig gespeichert und bei neuen Technologien erneut bewertet werden.
- 193 Opportunity Detector: JARVIS erkennt, wenn eine neue Technik eines deiner alten Projekte plötzlich möglich macht.
- 194 Future Capability Planner: Für noch unmögliche Funktionen hält er fest, welche technischen Voraussetzungen fehlen.
- 195 Personal R&D Lab: Eigener Bereich, in dem Agenten experimentelle Features nur in Sandbox testen.
- 196 Self-Benchmarking: Neue JARVIS-Version gegen die alte Version testen: schneller? zuverlässiger? besser?
- 197 A/B Personality Testing: Zwei Varianten eines Verhaltens testen und deine Präferenz lernen.
- 198 Continuous Improvement Reports: Monatlich zeigen, worin JARVIS messbar besser geworden ist.
- 199 Evolution Without Core Mutation: Er kann Skills, Modelle, Routinen und Strategien verbessern, ohne eigenmächtig den Sicherheitskern umzuschreiben.
- 200 JARVIS Identity Persistence: Selbst nach Jahren, Hardwarewechseln und Modellupdates bleiben Memory, Stil, Fähigkeiten, Beziehungen, Historie und Persönlichkeit konsistent.

28. WEITERE „KRASSE“ FEATURES AUS DEM MASTER-KONZEPT

- Teach Mode: „Merk dir, was ich jetzt mache“ -> Workflow-Demonstration -> abstrahierter Skill -> Test -> Freigabe.
- Workspace Reconstruction: Arbeitsumgebung von gestern inklusive Apps/Fenstern/Projekt/Terminals wiederherstellen.
- Personal Knowledge Graph: Projekt -> Person -> Datei -> Entscheidung -> Meeting -> Task verknüpfen.
- Time Machine UI: Projekt-/News-/Systemzustände entlang einer Zeitachse rekonstruieren.
- Reality Check Agent: eigener unabhängiger Verifier, dessen Job ausschließlich das Widerlegen falscher Erfolge ist.
- Simulation Before Action: vor riskanten Aktionen vollständige Vorschau der Änderungen und erwarteten Nebenwirkungen.
- Undo Everything soweit technisch möglich: Versionierung, Snapshots, Scenes und Release rollback.
- Network Command Center: Topologie deiner eigenen Geräte, Health, Bandbreite und Servicezustand.
- Follow-Me Presence: JARVIS folgt sinnvoll zwischen Geräten/Räumen, ohne überall gleichzeitig zu sprechen.
- Project Guardian: Tests, Backups, offene Bugs, Abhängigkeiten und Sicherheitszustand wichtiger Projekte beobachten.
- Personal Command Language: eigene Kurzbegriffe und Makros entstehen aus deiner Nutzung.
- Performance Guardian: JARVIS schützt seine eigene UX vor CPU/GPU/Netzwerküberlastung.
- Offline Degradation: kein Internet -> lokale Voice/Home/Device/Memories bleiben funktional.
- Digital Forensics: „Warum ist der PC gestern ausgegangen?“ -> Events/Logs/Temperaturen/Updates als Timeline.
- Opportunity Re-evaluation: alte Ideen werden bei neuen Hardware-/AI-Fähigkeiten erneut bewertet.

29. ABNAHMEKRITERIEN FÜR „JARVIS 1.0“

Dimension	1.0 muss mindestens
Reliability	24h Home-Core-Betrieb, definierter Recovery-Pfad, keine Mission geht still verloren.
Voice	Wake + duplex-ish conversation + barge-in + clean TTS + local fast path.
Desktop	Apps/Files/Browser/Terminal in erlaubtem Scope + screen context + verifier.
Mobile	Remote command, notification, approval und shared mission state.
Home	Mindestens Home Assistant + mehrere Geräteklassen + offline local actions.
Memory	Search, correction, forget, source/confidence, user profile.
Agent	Research/Coding/Verification-Agenten, budgets, graceful failure.
UI	Central core, mission mode, coding mode; World Globe funktional; fluidity budgets gemessen.
Security	P0-P6, passkey/biometric critical approvals, audit, secret isolation, kill switch.
Distribution	Private signed installer/release, updater, rollback, encrypted backup/restore.
Personality	konsistente eigene Stimme/Persona, Sarkasmus- und Humorregeln, no-filler.
Ownership	funktioniert als private Installation; persönliche Daten nicht im GitHub-Repo.

30. WAS NOCH VOR DER ECHTEN IMPLEMENTIERUNG ENTSCHEIDEN WIR

Diese Punkte fehlen im Chat noch. Sie blockieren das Konzept nicht, müssen aber spätestens vor den jeweiligen Phasen festgelegt werden:

- ☐ Haupt-PC: Windows-Version, Mainboard/Netzwerkkarte und Wake-on-LAN-Support.
- ☐ Handy: Android oder iPhone; bestimmt Always-on-Wake-Strategie.
- ☐ Welche Alexa/Echo-Modelle vorhanden sind.
- ☐ Welche Smart-Home-Geräte/Hersteller bereits existieren.
- ☐ Ob ein separater Home-Core/Mini-PC gekauft wird oder vorhandene Hardware genutzt wird.
- ☐ Ob lokale Kameraverarbeitung gewünscht ist und welche Privacy-Retention gilt.
- ☐ Welche TTS-Stimme als originale JARVIS-Identität ausgewählt/trainiert/lizenziert wird.
- ☐ Welche Aktionen P3/P4/P5 genau brauchen und welche immer verboten bleiben.
- ☐ Backup-Ziel: NAS, externe SSD, verschlüsselter Cloud-Backup oder Kombination.
- ☐ Budget für Claude API/weitere Voice-/News-Dienste im späteren Dauerbetrieb.

31. SOURCE OF TRUTH / HANDOFF-PROTOKOLL

WENN DIESE PDF SPÄTER HOCHGELADEN WIRD Behandle sie als Master-Spezifikation. Frage zuerst nach dem aktuellen Repository-/Versionsstand und dem zuletzt abgeschlossenen Phase-Exit-Kriterium. Ändere Kernarchitektur nur mit expliziter Begründung/ADR. Halte neue Entscheidungen als Delta zur PDF fest.

Empfohlene Statusdatei im Repository

```
docs/STATUS.md

Current version: 0.x.y
Current phase: PHASE N
Last completed milestone: ...
Active branch: ...
Known blockers: ...
Open security findings: ...
Next exact task: ...
Last benchmark: ...
Last backup/restore drill: ...
```

32. QUELLEN UND AKTUELLE TECHNISCHE ANNAHMEN

Research-Stand: 19.08.2026. Quellen belegen externe Plattformfähigkeiten; die Architektur- und Produktentscheidungen in diesem Blueprint sind das Projektdesign.

[S1] Claude Pricing - Max 20x, Claude Code, plan usage. <https://claude.com/pricing> - Max bietet 5x oder 20x Pro-Nutzung; Claude Code ist enthalten.

[S1A] Use the Claude Agent SDK with your Claude plan. <https://support.claude.com/de/articles/15036540-verwenden-sie-das-claude-agent-sdk-mit-ihrem-claude-plan> - Aktuelle Agent-SDK-Credit-Angaben für berechnete Pläne; Bedingungen können sich ändern.

[S2] Claude Platform Release Notes - Opus 5. <https://platform.claude.com/docs/en/release-notes/overview> - Opus 5 Launch 24 July 2026, 1M context, 128k output, effort controls.

[S2A] What's new in Claude Opus 5. <https://platform.claude.com/docs/en/about-claude/models/whats-new-claude-4-6> - Long-horizon, agentic coding, multi-agent, effort max.

[S3] Claude Agent SDK overview. <https://code.claude.com/docs/en/agent-sdk/overview> - Agent loop and built-in file/command/edit capabilities in Python/TypeScript.

[S4] Claude Agent SDK quickstart. <https://code.claude.com/docs/en/agent-sdk/quickstart> - SDK setup and authentication guidance.

[S5] Claude Managed Agents - multiagent orchestration. <https://platform.claude.com/docs/en/managed-agents/multiagent-orchestration> - Context-isolated agent threads and coordination.

[S6] Claude Code permissions / hooks. <https://code.claude.com/docs/en/permissions> - PreToolUse hooks and deny/ask/allow behavior.

[S7] Model Context Protocol 2026-07-28 specification announcement. <https://blog.modelcontextprotocol.io/posts/2026-07-28/> - Stateless core, routing, caching, authorization hardening, Tasks extension.

[S8] Home Assistant 2026.3 - Android wake word. <https://www.home-assistant.io/blog/2026/03/04/release-20263/> - On-device microWakeWord on Android; Hey Jarvis option and battery considerations.

[S9A] Home Assistant approach to wake words. https://www.home-assistant.io/voice_control/about_wake_word/ - Wake-word architecture and microWakeWord models.

[S9] Tauri 2. <https://tauri.app/> - Cross-platform desktop and mobile apps from a shared codebase.

[S10] Three.js WebGPURenderer. <https://threejs.org/manual/en/webgpurenderer> - WebGPU renderer with WebGL 2 fallback.

[S11] GitHub Docs - About repositories. <https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories> - Private repository visibility and access.

[S12] GitHub Docs - About releases. <https://docs.github.com/en/repositories/releasing-projects-on-github/about-releases> - Versioned release assets and read-access behavior.

33. MASTER-ENTSCHEIDUNG

BUILD ORDER Wir beginnen NICHT mit Alexa, 3D-Erde oder Sprach-Sarkasmus. Wir beginnen mit Phase 0 und dann Phase 1: JARVIS Core - Event Bus, Mission State, Capability Registry, Permission Engine, Execution Gateway und Verifier. Sobald dieses Gehirn korrekt handelt, bekommt es Memory, Claude, Voice, Desktop, Home, Mobile und danach das volle futuristische Interface.

Wenn dieser Kern sauber gebaut ist, kann fast jedes spätere „krasse Feature“ als Capability, Adapter, Agent oder UI-Mode ergänzt werden. Wenn der Kern schlecht gebaut ist, werden 200 Features nur 200 zusätzliche Fehlerquellen. Dieses Blueprint priorisiert deshalb langfristige Erweiterbarkeit, Sicherheit und Fluidität über schnelle Show-Demos.